



PROGRAMAÇÃO



Escola Naval



António Gonçalves

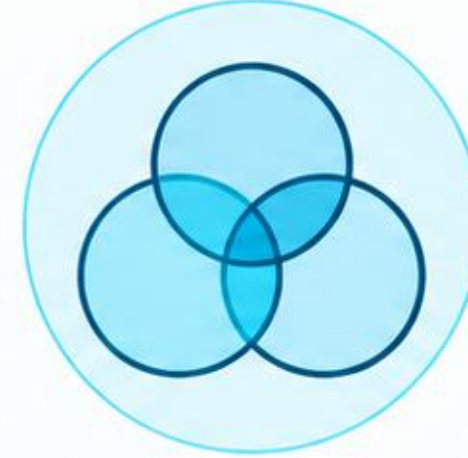
Agenda – Seleção II

Decisões múltiplas



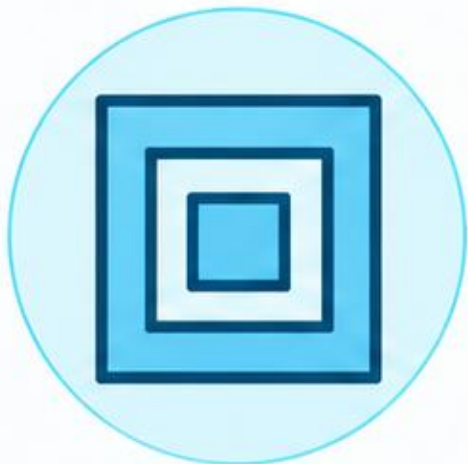
elif e ordem das condições

Condições compostas



and, or, not e parênteses

Decisões aninhadas



Uma decisão dentro de outra

Validação e testes



Dados válidos e casos-limite

Revisão: duas alternativas

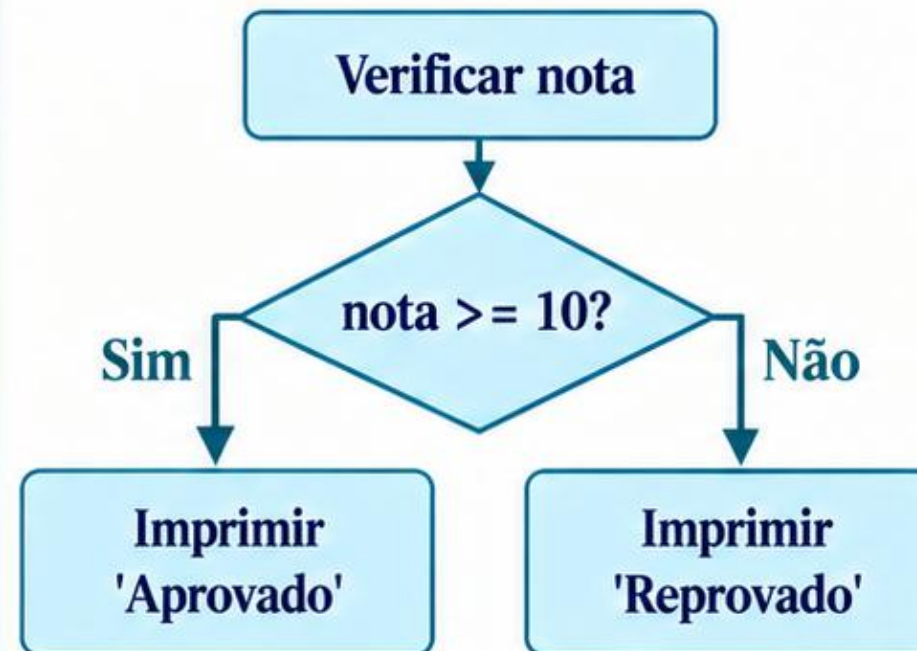
Código em Python

O programa decide entre duas alternativas: aprovado ou reprovado.

```
nota = 12
if nota >= 10:
    print('Aprovado')
else:
    print('Reprovado')
```

Como funciona?

O programa verifica se a nota é maior ou igual a 10.



Resultados de exemplo

Com diferentes valores de nota:

nota	Resultado	Explicação
9	Reprovado	9 é menor do que 10.
10	Aprovado	10 é maior ou igual a 10.
12	Aprovado	12 é maior do que 10.



Ideia principal: Esta estrutura `if/else` produz apenas duas alternativas. Para classificar números negativos, zero e positivos não podemos expressar todas as três situações num único `if/else`. A seguir vamos introduzir o `elif`. Suponha que a nota está no intervalo 0..20.

Alternativas múltiplas com elif

O que faz

Permite escolher uma de várias alternativas, testando condições por ordem, até encontrar a primeira que é verdadeira.

Ler dados



Testar condições
(if / elif / else)



Executar a ação
correspondente

Exemplo em Python

```
numero = int(input('Número: '))  
if numero > 0:  
    print('Positivo')  
elif numero < 0:  
    print('Negativo')  
else:  
    print('Zero')
```

Como funciona

As condições são verificadas de cima para baixo, e apenas a primeira que é verdadeira é executada.

Resultados de teste

Número introduzido	Resultado
-2	Negativo
0	Zero
5	Positivo



Ideia principal:

Exatamente um dos ramos (if, elif ou else) é executado.

Ordem de avaliação

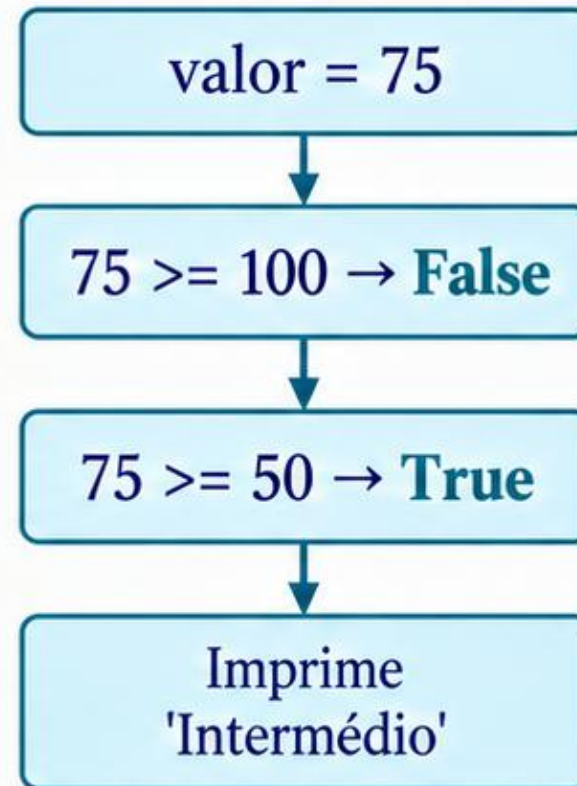
Código e objetivo

O programa avalia o valor da variável e apresenta uma classificação.

```
valor = 75
if valor >= 100:
    print('Elevado')
elif valor >= 50:
    print('Intermédio')
else:
    print('Baixo')
```

Como é avaliado

As condições são verificadas de cima para baixo. Para valor = 75:



Resultados de testes

A tabela mostra o resultado do programa para diferentes valores.

Valor	Resultado
25	Baixo
50	Intermédio
75	Intermédio
100	Elevado



Após a primeira condição verdadeira, as restantes condições não são avaliadas.

A ordem das condições altera o resultado

Regra de classificação

As classificações dependem da nota (entre 0 e 20) e seguem estas condições:

Condição	Classificação
Nota ≥ 16	Muito bom
Nota ≥ 10	Aprovado
Caso contrário	Reprovado

A nota pode variar entre 0 e 20, inclusive.

Código incorreto

Neste código, a condição mais baixa é verificada primeiro.

```
if nota >= 10:
    print('Aprovado')
elif nota >= 16:
    print('Muito bom')
else:
    print('Reprovado')
```

Para uma nota de 18, o programa imprime "Aprovado", o que está incorreto.

Código correto

Neste código, a condição mais exigente é verificada primeiro.

```
if nota >= 16:
    print('Muito bom')
elif nota >= 10:
    print('Aprovado')
else:
    print('Reprovado')
```

Para uma nota de 18, o programa imprime "Muito bom", o que está correto.



Ideia principal:

As condições devem ser testadas da mais exigente para a menos exigente.

If independentes e cadeia elif

If independentes

Os if são avaliados de forma independente. Podem ser executadas várias instruções, se todas as condições forem verdadeiras.

Código Python

```
nota = 18

if nota >= 10:
    print('Aprovado')
if nota >= 16:
    print('Muito bom')
```

Resultado

Aprovado
Muito bom

Cadeia elif

Na cadeia if...elif... só é executado o primeiro bloco cuja condição seja verdadeira. As categorias são exclusivas.

Código Python

```
nota = 18

if nota >= 16:
    print('Muito bom')
elif nota >= 10:
    print('Aprovado')
```

Resultado

Muito bom



Escolhe if independentes quando as mensagens são cumulativas; usa a cadeia elif quando as categorias são exclusivas. Nenhuma das abordagens é sempre melhor.

Exercício: classificação por escalões

Regras de classificação

A nota é um número inteiro entre 0 e 20. A classificação é atribuída de acordo com os seguintes escalões:

< 10 → Reprovado

10 a 13 → Suficiente

14 a 17 → Bom

18 a 20 → Muito bom

São consideradas apenas notas inteiras.

Código Python

```
if nota >= 18:
    classificacao = 'Muito bom'
elif nota >= 14:
    classificacao = 'Bom'
elif nota >= 10:
    classificacao = 'Suficiente'
else:
    classificacao = 'Reprovado'
```

Exemplos de resultados

Nota	Classificação
9	Reprovado
10	Suficiente
13	Suficiente
14	Bom
17	Bom
18	Muito bom
20	Muito bom



Ideia principal:

Estruturas condicionais permitem atribuir categorias a intervalos de valores.

Operador lógico and

Condição com duas condições verdadeiras

O operador **and** só devolve **True** quando as duas condições são verdadeiras.

Exemplo:

```
idade = 20
tem_bilhete = True

if idade >= 18 and tem_bilhete:
    print('Entrada autorizada')
```

Tabela de verdade

O resultado é **True** apenas quando ambos os operandos são **True**.

Operando 1	Operando 2	Resultado (and)
False	False	False
False	True	False
True	False	False
True	True	True

Os operandos desta tabela são valores booleanos.

Resultados de testes

O operador **and** requer as duas condições verdadeiras.

Idade	Tem bilhete	Condição (idade >= 18 and tem_bilhete)	Resultado
20	True	True	Entrada autorizada
17	True	False	
20	False	False	



Ideia principal:

O operador **and** só é verdadeiro quando as duas condições são verdadeiras.

Operador lógico or

Pelo menos uma condição verdadeira

O operador lógico **or** devolve **True** se pelo menos uma das condições for verdadeira.

Tabela de verdade

Condição 1	Condição 2	Resultado (com or)
False	False	False
False	True	True
True	False	True
True	True	True

Exemplo

A expressão é verdadeira porque pelo menos uma das variáveis é verdadeira.

```
tem_convite = False
tem_bilhete = True

if tem_convite or tem_bilhete:
    print('Pode entrar')
```



Ideia principal:

O operador **or** é inclusivo: se ambas as condições forem verdadeiras, o resultado também é verdadeiro.

Operador lógico not

O que faz o operador not

O operador not inverte o valor lógico de uma condição.



True → False
False → True

Exemplo em Python

O operador not é frequentemente usado para negar uma condição.

```
autorizado = False  
if not autorizado:  
    print('Acesso recusado')
```

Tabela de verdade

O operador not inverte o valor lógico.

Valor original	not (valor)
True	False
False	True

Negar uma condição

Para negar toda a condição, utilizam-se parênteses.

```
not (nota >= 10)
```

Equivalência

A condição not (nota >= 10) é equivalente a:

```
nota < 10
```

Ideia principal



O operador not inverte o valor lógico de uma condição.

Intervalos e condições compostas

Valores válidos: de 0 a 20

Uma nota é válida se estiver no intervalo de 0 a 20, inclusive.

```
if nota >= 0 and nota <= 20:  
    print("Nota válida!")
```

Forma equivalente:

```
if 0 <= nota <= 20:  
    print("Nota válida!")
```



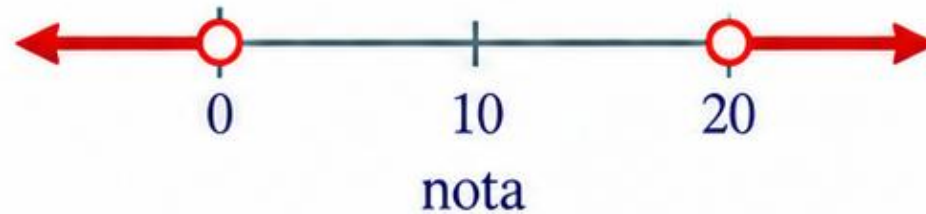
Valores fora do intervalo

Uma nota é inválida se for inferior a 0 ou superior a 20.

```
if nota < 0 or nota > 20:  
    print("Nota inválida!")
```

Forma equivalente:

```
if not (0 <= nota <= 20):  
    print("Nota inválida!")
```



Resultados dos testes

Nota	Válida (0 a 20)	Inválida (fora do intervalo)
-1	False	True
0	True	False
10	True	False
20	True	False
21	False	True

Erro comum

A condição `nota < 0 and nota > 20` é impossível para um número real. Um valor não pode ser, ao mesmo tempo, menor que 0 e maior que 20.



Ideia principal:

Usam-se condições compostas (**and**, **or**, **not**) para testar intervalos de valores. A comparação encadeada `0 <= nota <= 20` é equivalente à condição com **and**.

Precedência e parênteses

Precedência de operadores

Em Python, o operador `and` tem maior precedência do que o operador `or`. Por isso, a expressão é avaliada primeiro com o `and` e depois com o `or`.

Precedência: comparações → `not` → `and` → `or`.

```
idade = 16
tem_bilhete = False
tem_convite = True
```

```
idade >= 18 and tem_bilhete or tem_convite => True
(idade >= 18 and tem_bilhete) or tem_convite => True
idade >= 18 and (tem_bilhete or tem_convite) => False
```

Resultados das expressões

Expressão	Resultado
<code>idade >= 18 and tem_bilhete or tem_convite</code>	True
<code>(idade >= 18 and tem_bilhete) or tem_convite</code>	True
<code>idade >= 18 and (tem_bilhete or tem_convite)</code>	False

Interpretação dos resultados

Nas duas primeiras expressões, o convite permite a entrada independentemente da idade.

Na terceira expressão, é necessário ter pelo menos 18 anos para qualquer tipo de bilhete (normal ou convite).



Ideia principal:

Utilize parênteses explícitos para tornar clara a sua intenção.

Avaliação em curto-circuito

Operador **and** (stop first False)

Se a primeira condição for falsa, a segunda não é avaliada.

Exemplo:

```
divisor = 0  
if divisor != 0 and 10 / divisor > 2:  
    print("Quociente superior a 2")
```

A segunda condição não é avaliada, portanto não ocorre um erro de divisão por zero.



Operador **or** (stop first True)

Se a primeira condição for verdadeira, a segunda não é avaliada.

Exemplo:

```
nome = ''  
if nome == '' or nome[0] == 'A':  
    print("Vazio ou começa por A")
```

A segunda condição não é avaliada, portanto não ocorre um erro de índice (IndexError).



Ideia principal:

A avaliação em curto-circuito evita avaliar expressões desnecessárias.

Decisões aninhadas

Código em Python

Uma decisão pode estar dentro de outra decisão.

```
idade = 20
tem_bilhete = False
if idade >= 18:
    if tem_bilhete:
        print('Entrada autorizada')
    else:
        print('Falta bilhete')
else:
    print('Idade insuficiente')
```

Como funciona

Primeiro é verificada a idade.
Se a idade for maior ou igual a 18,
é então verificado se tem bilhete.
A verificação do bilhete só acontece
se a idade for aceite.

Se idade >= 18:

Se tem_bilhete:

Entrada autorizada

Caso contrário:

Falta bilhete

Caso contrário:

Idade insuficiente

Resultados dos testes

Os seguintes testes mostram
os diferentes resultados.

idade	tem_bilhete	Resultado
17	True	Idade insuficiente
20	False	Falta bilhete
20	True	Entrada autorizada



Ideia principal:

As decisões podem ser aninhadas para tratar várias condições.

Indentação e associação do else

Código e estrutura

A indentação determina a que if cada else está associado.

```
if idade >= 18:
    if tem_bilhete:
        print("Entrada autorizada")
    else:
        print("Falta bilhete")
else:
    print("Idade insuficiente")
```

Níveis de indentação

Nível exterior:
if e else começam na mesma coluna.

Nível interior:
o segundo if e o seu else estão recuados 4 espaços.

Instruções:
os print estão recuados 4 ou 8 espaços, conforme o bloco.

Exemplos de execução

Exemplo 1

```
idade = 17
tem_bilhete = False
```



Resultado:

Idade insuficiente

Exemplo 2

```
idade = 20
tem_bilhete = False
```



Resultado:

Falta bilhete



Ideia principal:

Cada else pertence ao if que está no mesmo nível de indentação.

Aninhar ou combinar condições

Condições aninhadas

Verifica primeiro a idade. Se for maior ou igual a 18, verifica o bilhete.

idade >= 18

tem_bilhete

Ambas verdadeiras:
imprime
Entrada autorizada

Código Python

```
if idade >= 18:  
    if tem_bilhete:  
        print('Entrada autorizada')
```

Em caso de falha

Se alguma condição for falsa, este código não imprime qualquer mensagem.

Uma única condição combinada

Verifica, numa só condição, se a idade é maior ou igual a 18 e se existe bilhete.

**idade
>= 18**

AND

tem_bilhete

Ambas verdadeiras:
imprime
Entrada autorizada

Código Python

```
if idade >= 18 and tem_bilhete:  
    print('Entrada autorizada')
```

Em caso de falha

Se alguma condição for falsa, este código não imprime qualquer mensagem.



Ideia principal:

Usa **and** quando tens um único requisito conjunto.
Aninha condições quando precisas de tratar razões ou ações diferentes.

Validação antes da classificação

Programa em Python

O valor introduzido é convertido para inteiro e validado antes da classificação.

```
nota = int(input('Nota inteira: '))
if nota < 0 or nota > 20:
    print('Nota inválida')
elif nota >= 18:
    print('Muito bom')
elif nota >= 14:
    print('Bom')
elif nota >= 10:
    print('Suficiente')
else:
    print('Reprovado')
```

Resultados da classificação

Nota introduzida	Resultado
-1	Nota inválida
0	Reprovado
10	Suficiente
14	Bom
18	Muito bom
20	Muito bom
21	Nota inválida

Notas importantes

Valores válidos

São aceites valores inteiros entre 0 e 20, inclusive.

Validação antes da classificação

Se a nota for menor que 0 ou maior que 20, é apresentada a mensagem “Nota inválida”.

Erros de conversão

Se o valor introduzido não puder ser convertido para inteiro, ocorrerá um erro de conversão. O tratamento desse erro será abordado mais à frente na aula sobre exceções.



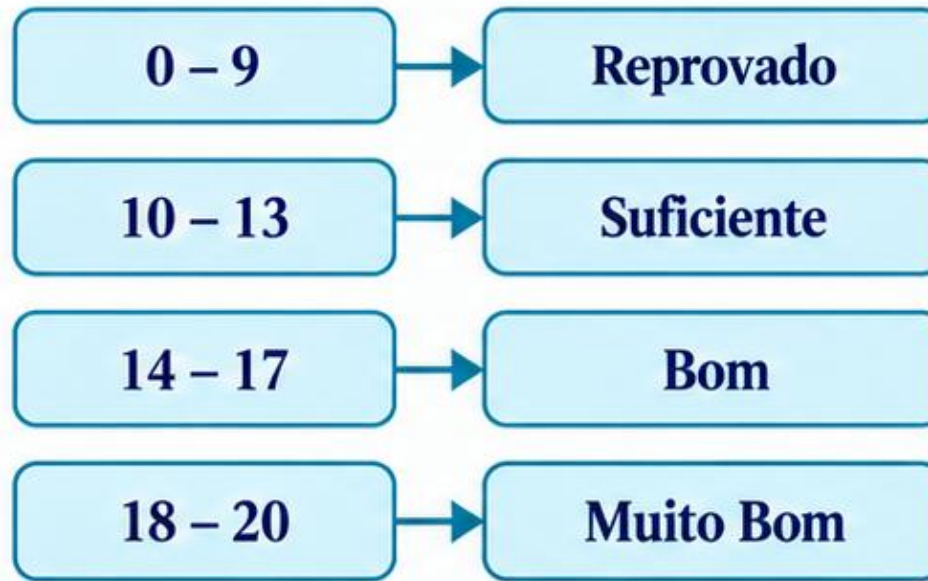
Ideia principal:

Validar os dados antes de aplicar as regras de classificação.

Casos-limite e cobertura de testes

Classificação da nota

O programa recebe um número inteiro e atribui uma classificação de acordo com os intervalos definidos.



Valores fora do intervalo [0, 20] são considerados inválidos.

Casos de teste

Os testes incluem os valores-limite, um valor normal e valores fora do domínio válido, cobrindo todos os ramos de decisão.

Entrada	Resultado esperado
-1	Inválido
0	Reprovado
9	Reprovado
10	Suficiente
12	Suficiente
13	Suficiente
14	Bom
17	Bom
18	Muito Bom
20	Muito Bom
21	Inválido

Comparação de resultados

Para cada caso de teste, compara-se o resultado observado com o resultado esperado.

Entrada	Esperado	Observado	Teste
-1	Inválido	Inválido	OK
0	Reprovado	Reprovado	OK
9	Reprovado	Reprovado	OK
10	Suficiente	Suficiente	OK
12	Suficiente	Suficiente	OK
13	Suficiente	Suficiente	OK
14	Bom	Bom	OK
17	Bom	Bom	OK
18	Muito Bom	Muito Bom	OK
20	Muito Bom	Muito Bom	OK
21	Inválido	Inválido	OK



Ideia principal:

Os casos-limite e a comparação entre resultados esperados e observados garantem uma boa cobertura de testes.

Erros frequentes em condições

Comparação com vários valores

A expressão usa “or” de forma incorreta. O número 2 é considerado verdadeiro, logo a condição é sempre verdadeira.

✘ **Incorreto:**

```
if opcao == 1 or 2:  
    print("Opção válida")
```

✔ **Correto:**

```
if opcao == 1 or opcao == 2:  
    print("Opção válida")
```

Intervalo de valores

A expressão usa “or” de forma incorreta. A condição é verdadeira para qualquer número real.

✘ **Incorreto:**

```
if 0 <= nota or nota <= 20:  
    print("Nota válida")
```

✔ **Correto:**

```
if 0 <= nota and nota <= 20:  
    print("Nota válida")
```

Ordem dos testes

A ordem das condições é importante. Se o teste de nota ≥ 10 vier antes, uma nota 18 é classificada como "Aprovado".

✘ **Incorreto:**

```
if nota >= 10:  
    print("Aprovado")  
elif nota >= 18:  
    print("Excelente")
```

✔ **Correto:**

```
if nota >= 18:  
    print("Excelente")  
elif nota >= 10:  
    print("Aprovado")
```

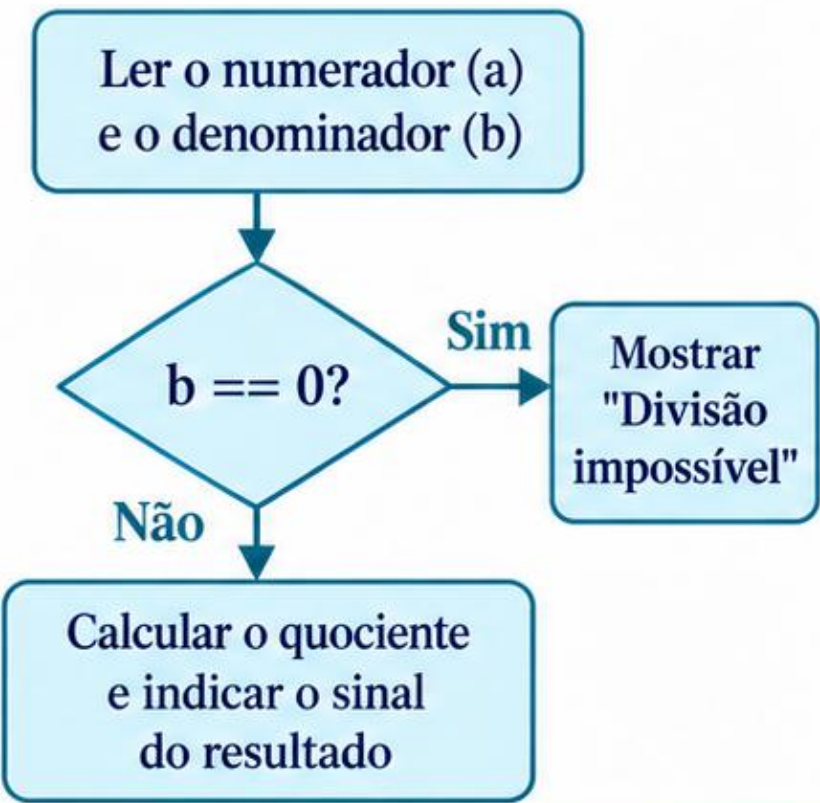


Pequenas diferenças nas condições podem alterar o resultado. Verifica sempre com atenção a lógica das expressões.

Aplicação: divisão com validação

Lógica do programa

A divisão só é realizada se o denominador não for zero.



Código Python

O programa lê o numerador e o denominador, verifica se o denominador é zero e, caso não seja, calcula o quociente e indica o seu sinal.

```
a = float(input('Numerador: '))
b = float(input('Denominador: '))
if b == 0:
    print('Divisão impossível')
else:
    resultado = a / b
    if resultado > 0:
        print('Quociente positivo:', resultado)
    elif resultado < 0:
        print('Quociente negativo:', resultado)
    else:
        print('Quociente nulo')
```

Resultados de testes

Com entradas numéricas válidas, obtêm-se os seguintes resultados:

Numerador (a)	Denominador (b)	Resultado indicado
6	2	Quociente positivo: 3.0
-6	2	Quociente negativo: -3.0
0	2	Quociente nulo
6	0	Divisão impossível



Ideia principal:

Validar o denominador evita uma divisão impossível e permite interpretar o sinal do quociente.

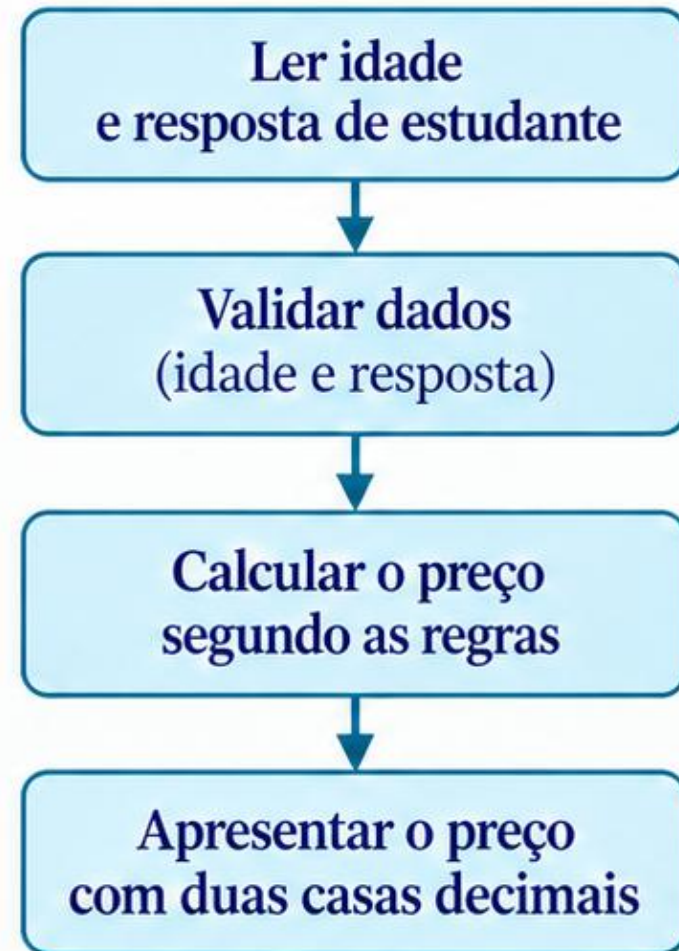
Exercício: bilhete com descontos

Regras do problema

Pede-se um programa que leia a idade e se o cliente é estudante, valide os dados, calcule o preço do bilhete e apresente o resultado com duas casas decimais.

- **Idade válida:** inteiro entre 0 e 120 (inclusive).
- **Preço base:** 12 euros.
- **Idade < 12:** paga 6 euros.
- **Idade entre 12 e 64 (inclusive):** paga 12 euros.
- **Idade ≥ 65:** paga 8 euros.
- **Desconto de estudante:** adicional de 2 euros, aplicado apenas para idades entre 12 e 64 (inclusive) e se a resposta for s.
- **Respostas válidas:** 's' ou 'n', normalizadas com `strip().lower()`. Qualquer outra resposta é inválida.

Processo geral



Exemplos de testes

Idade	Estudante	Resultado esperado
11	n	6.00
12	s	10.00
64	n	12.00
65	s	8.00
-1	n	Dados inválidos
30	x	Dados inválidos



Ideia principal:

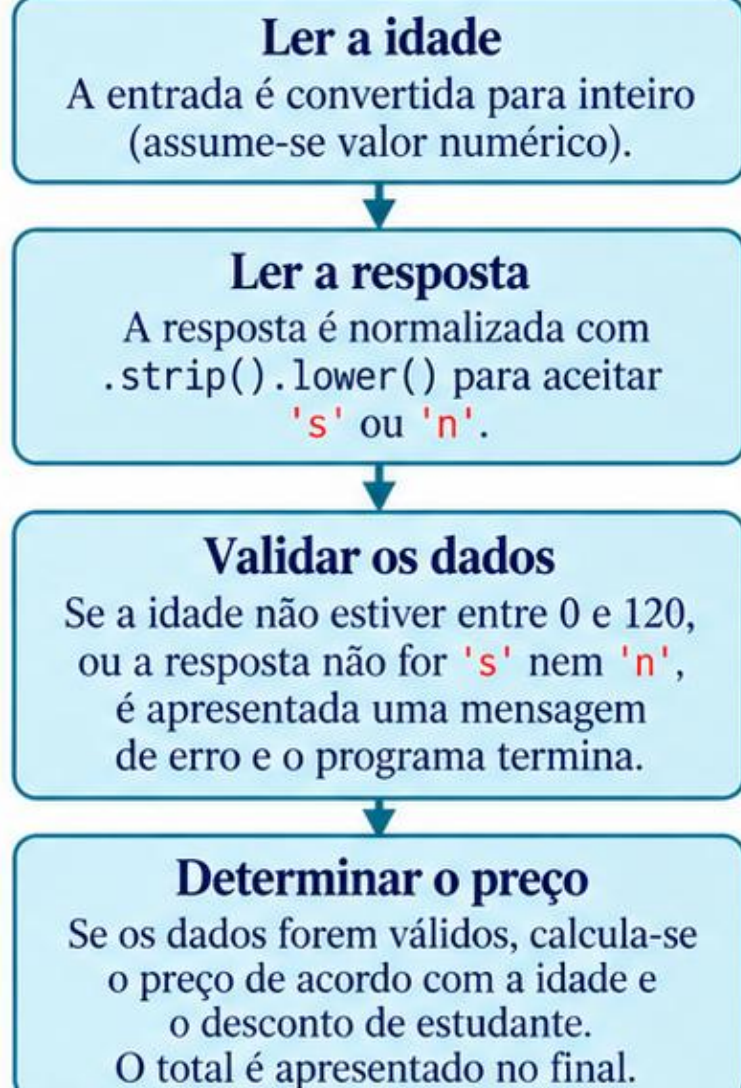
Usar condições para determinar o preço e validar sempre os dados de entrada.

Solução: validação e preço

Código em Python

```
idade = int(input('Idade: '))
resposta = input('Estudante (s/n): ').strip().lower()
if idade < 0 or idade > 120:
    print('Idade inválida')
elif resposta != 's' and resposta != 'n':
    print('Resposta inválida')
else:
    if idade < 12:
        preco = 6.0
    elif idade >= 65:
        preco = 8.0
    else:
        preco = 12.0
    if resposta == 's':
        preco -= 2.0
    print(f'Total: {preco:.2f} euros')
```

Como funciona



Exemplos de execução

Idade 12, estudante 's'

Idade: 12
Estudante (s/n): s
Total: 10.00 euros

Idade 65, estudante 's'

Idade: 65
Estudante (s/n): s
Total: 8.00 euros

Idade intermédia 12–64, estudante 's'

Idade: 30
Estudante (s/n): s
Total: 10.00 euros



Ideia principal:

Validar os dados primeiro e, se forem válidos, calcular o preço e apresentá-lo no final.

Síntese: decisões e validação

elif escolhe primeira condição verdadeira

As condições são avaliadas pela ordem apresentada e executa-se o primeiro bloco cuja condição é verdadeira.

Testar primeira condição



Se falsa, testar seguinte condição



Executar o primeiro bloco verdadeiro

and, or e not

Os operadores lógicos combinam condições:

and

Exige que ambas as condições sejam verdadeiras.

or

Admite que uma ou ambas as condições sejam verdadeiras.

not

Inverte o valor lógico de uma condição.

Esquema conceptual

Condições podem ser aninhadas quando uma decisão depende de outra.

if condição exterior:

if condição interior:

...

else:

...

Validar antes de calcular

É importante validar os dados antes de realizar cálculos e testar os valores de fronteira (limites) para detetar erros.

Questão

Qual é a saída deste código?

```
x = 10
if x < 0:
    print('A')
elif x <= 10:
    print('B')
else:
    print('C')
```



Confirmação final:

Resposta: **B**, porque $10 < 0$ é falso e $10 \leq 10$ é verdadeiro.

No próximo tema, veremos o ciclo **while**, que permite repetir instruções.